

DPDK BenchOps Copilot (GenAI RAG + Agentic Workflows)

Project / Team Name

Project: DPDK BenchOps Copilot **Organization:** Infobell IT Labs (engagement for AMD – CPU performance benchmarking) **Domain:** AI-assisted benchmark operations platform for high-performance network packet processing **Role:** Lead Developer / Senior AI Platform Engineer **Dates:** Evolved as the next stage of the DPDK benchmark automation platform after the core automation system matured

1. Business Opportunity (AI Evolution of AMD DPDK Benchmark Automation)

Business goal – move from benchmark automation to benchmark intelligence

After the team built a strong AMD-centric automation framework for DPDK and networking benchmarks, the next business opportunity was to solve a more difficult and higher-value problem: **scaling benchmark expertise.**

The original automation platform had already improved repeatability and reduced manual effort, but performance engineers still had to spend significant time answering complex questions such as:

- Which tuning guidance applies to this workload on this AMD server generation?
- Why did performance regress across BIOS / OS / compiler / hardware changes?
- Which previous runs are truly comparable?
- What parameters should be used for a new benchmark campaign?
- How should benchmark results, logs, and tuning guidance be connected into one trustworthy answer?

AMD's benchmarking workflows were no longer only an execution problem. They had become a **decision-making and knowledge-access problem.**

This created an opportunity to evolve the framework into a **production-style AI BenchOps Copilot** that could help engineers:

- ask benchmark and tuning questions in natural language
 - retrieve grounded answers with citations
 - generate validated benchmark run plans
 - construct safe benchmark commands through deterministic tools
 - parse and compare results across runs
 - improve productivity without weakening correctness or safety
-

Problem 1 – Knowledge was rich, but fragmented across tools and formats

The automation platform had already created valuable assets such as:

- AMD-centric benchmark documentation
- benchmark run configurations
- parsed benchmark logs
- performance metrics stored in structured form
- comparison dashboards

- internal runbooks and tuning notes
- OS / compiler / BIOS variation data
- workload-specific templates for DPDK crypto, testpmd, vhost, L2/L3 forwarding, and related scenarios

However, this knowledge remained fragmented.

Engineers still had to manually correlate:

- benchmark guides
- dashboard views
- raw logs
- run metadata
- BIOS configuration context
- performance deltas across multiple runs

As a result, the platform could execute and report benchmarks well, but it still relied heavily on experienced engineers to interpret the results correctly.

Problem 2 – Traditional automation reduced effort, but not cognitive load

The first generation framework solved major execution problems:

- setup and environment consistency
- benchmark orchestration
- BIOS profile switching
- stats collection
- parsing and reporting
- multi-server scenario execution

But it did not fully solve the next-layer questions:

- “What should I run next?”
- “Why did this run regress?”
- “What tuning guidance is relevant here?”
- “What does this metric actually mean?”
- “Which command variation is correct for this scenario?”
- “What is documented fact vs hypothesis?”

That meant benchmark workflows still had a large amount of **human cognitive overhead**, especially when dealing with:

- multiple AMD server generations
- different benchmark types
- changing BIOS recommendations
- many workload-specific command parameters
- noisy performance data across releases

Problem 3 – AI could help, but hallucination and unsafe execution were unacceptable

The team saw clear value in adding AI assistance, but the domain had a very high trust requirement.

A generic chatbot approach would not be safe enough because it could:

- hallucinate tuning recommendations
- misinterpret DPDK parameters
- generate invalid benchmark commands
- overgeneralize guidance from one platform to another
- recommend disruptive actions like BIOS changes without proper control

So there was a need for a carefully designed enterprise AI approach where:

- **all factual guidance must be grounded in retrieved benchmark knowledge**
 - **commands must be generated only by deterministic tools**
 - **high-risk actions like BIOS changes or reboot guidance must require explicit approval**
 - **responses must be auditable, explainable, and safe for production-style use**
-

Problem 4 – The platform needed to behave like a real internal AI backend, not a demo

This was not meant to be a lightweight chatbot layered on top of documents.

The goal was to create an internal AI capability that operated like a real enterprise backend system:

- connected to run data and artifacts
- integrated with benchmark logs and platform metadata
- backed by a structured storage model
- able to call deterministic tools safely
- deployable on Kubernetes
- observable and evaluable in CI/CD
- resilient to dependency failures and regressions

This was an opportunity to turn the benchmark framework into a **production-style AI platform capability** rather than a one-off automation utility.

2. My Developer Contribution – Role of Radheshyam

Lead role – evolving a deterministic benchmark platform into an AI-assisted enterprise system

Radheshyam drove the architectural evolution from a strong deterministic benchmark automation system into a **DPDK BenchOps Copilot**.

His contribution was not simply “adding AI on top.” He defined **where AI should help, where deterministic systems should remain in control, and how to combine both safely**.

Because **Radheshyam** had already led the original automation effort, he had deep understanding of:

- DPDK benchmark flows
- BIOS and platform variability
- command complexity
- structured parsing and reporting
- how engineers actually used the system in practice

That allowed him to design the AI layer in a way that was realistic, useful, and operationally safe.

a) Creating the right foundation for AI from the existing benchmark platform (Radheshyam)

Using the automation platform as the substrate for AI reasoning

One of the strongest parts of **Radheshyam's** contribution was that the AI system was built on top of a real benchmark foundation, not invented from scratch.

The earlier framework already included:

- automation for 7 networking benchmarks
- end-to-end implementation of complex workloads such as DPDK crypto, DPDK vhost, and DPDK testpmd
- reusable benchmark templates for commands with many variables
- stats collection pipelines
- custom parsers for raw benchmark outputs
- structured database-backed reporting
- comparison flows across BIOS / OS / compiler / CPU SKU changes
- AMD-centric runbooks and operational guidance

Radheshyam recognized that these assets were exactly what an AI system needed in order to be useful.

He used this deterministic benchmark platform as the **truth-bearing operational substrate** for the AI copilot.

That meant the AI layer was grounded in real:

- run history
- structured metrics
- benchmark documentation
- comparison records
- tuning notes
- platform metadata

rather than vague or generic benchmark advice.

b) Designing the AI architecture and reasoning boundaries (Radheshyam)

Defining how AI should be used safely in benchmark operations

A major architectural decision by **Radheshyam** was that the copilot would **not** be allowed to behave like a free-form assistant.

Instead, he designed a layered architecture where:

- **RAG was the source of truth** for factual benchmark/tuning guidance
- **LangGraph handled multi-step workflow orchestration**
- **LangChain acted as the LLM interface and tool-calling glue**
- **MCP provided a deterministic tool boundary**
- **Postgres / S3 / MinIO stored structured truth and artifacts**
- **Vector DB stored semantic representations for retrieval**
- **FastAPI exposed the service layer**

- **Kubernetes + Helm handled production-style deployment**

This design ensured that the system remained:

- grounded
- auditable
- safer than a generic LLM assistant
- aligned with enterprise backend expectations

Separating AI-assisted reasoning from deterministic execution

Radheshyam deliberately decided that the following should remain deterministic:

- benchmark command generation
- plan validation
- run comparisons
- log fetching
- result parsing
- disruptive operation gating

AI was used only where it added value:

- benchmark Q&A
- tuning guidance synthesis from evidence
- natural language to structured plan translation
- regression explanation support
- result summarization
- metric explanation

This separation became a key strength of the platform.

c) LlamaIndex-based ingestion and benchmark-aware retrieval (Radheshyam)

Turning fragmented knowledge into a searchable, grounded benchmark knowledge base

To support reliable RAG, **Radheshyam** implemented the ingestion pipeline using **LlamaIndex**.

The ingestion pipeline processed multiple source types such as:

- benchmark logs
- database JSON records
- AMD tuning documents
- benchmark methodology guides
- historical run metadata
- internal benchmark notes and references

Normalization and phase-aware chunking

A key design contribution from **Radheshyam** was that the pipeline did not use naive chunking.

Instead, he designed the pipeline to:

- normalize raw input formats

- break content into meaningful benchmark-aware chunks
- preserve phase-specific context such as setup, execution, metrics, and comparison interpretation
- extract metadata relevant to workload, platform, and source lineage

This made the retrieval system much more useful for performance engineering questions.

Metadata extraction for precise retrieval

Radheshyam ensured the indexed data carried benchmark-relevant metadata such as:

- benchmark type (CryptoPerf / testpmd / L3fwd / other workload tags)
- AMD server generation tags
- source type (document / log / DB record / run artifact)
- run context where applicable
- provenance needed for citations

This allowed the retrieval layer to support not just semantic similarity, but **context-aware retrieval**.

Truth storage vs semantic indexing

To keep the platform well-structured, **Radheshyam** designed the storage pattern so that:

- structured authoritative records and artifacts stayed in **Postgres and S3/MinIO**
- embeddings and semantic search lived in the **vector database**

This made the system easier to reason about and aligned with production backend design principles.

d) LangGraph + LangChain workflow orchestration (Radheshyam)

Moving from prompt chains to controlled workflows

Because the use case involved multiple steps and validation points, **Radheshyam** used **LangGraph** and **LangChain** to orchestrate the AI flows.

This was important because a single-shot prompt was not enough for benchmark operations.

The platform had to handle workflows such as:

1. identify the user's intent
2. retrieve relevant benchmark context
3. filter by metadata such as platform / workload / source type
4. call deterministic tools where needed
5. verify that the answer is supported by retrieved evidence
6. return the final answer with citations and structured outputs

Hybrid retrieval + tool usage

Radheshyam implemented orchestration so the copilot could combine:

- hybrid retrieval
- deterministic run and log access
- structured comparison flows
- response verification

This allowed the system to handle tasks like:

- answering tuning questions
- explaining benchmark regressions
- generating validated benchmark plans
- summarizing run comparisons
- explaining metrics with evidence

Verification before final response

An important part of the workflow defined by **Radheshyam** was that responses were not treated as valid just because the LLM produced them.

The workflow included a verification step to ensure the final answer was supported by retrieved context and tool results.

This significantly improved trust in the system.

e) MCP tool boundary for safe deterministic operations (Radheshyam)

Commands and operational actions stayed tool-driven

Radheshyam defined and integrated the **MCP tool boundary** to ensure that execution-sensitive functions stayed deterministic.

Rather than allowing the LLM to emit shell commands directly, the system exposed safe tools such as:

- **RunQuery** – fetch benchmark runs and structured run metadata
- **LogFetch** – retrieve logs and artifacts for analysis
- **RunDiff** – compare runs deterministically across metrics and configurations
- **CommandBuilder** – generate safe benchmark commands from allowlisted templates

Allowlisted command templates

This design was especially valuable because the benchmark domain was highly parameterized.

For example, DPDK crypto alone involved:

- 23+ commands
- 10+ variables per command
- workload-specific runtime combinations

Since **Radheshyam** had already converted many of these benchmark flows into reusable templates in the deterministic platform, he could now expose them safely through MCP instead of relying on free-form LLM generation.

Audit logging and tool safety

Radheshyam ensured that tool calls were controlled and auditable, which improved:

- safety
- reproducibility
- operator trust
- debugging ability

Manual gate for BIOS / reboot-affecting actions

Since BIOS and reboot-related changes were already known to be high-risk operational steps from the original framework, **Radheshyam** explicitly kept these behind human-controlled safety boundaries.

The copilot could surface guidance, but not execute such actions automatically.

That was a deliberate production safety decision.

f) Plan generation, regression analysis, and benchmark question answering (Radheshyam)

Natural-language to benchmark workflow assistance

One of the key goals of the BenchOps Copilot was to let engineers interact with the benchmark platform at a higher level.

Using the AI layer designed by **Radheshyam**, engineers could:

- ask natural-language questions about tuning and regressions
- retrieve grounded answers with citations from run/log/guide context
- generate benchmark run plans from natural-language goals
- build safer structured commands through deterministic tool flows
- compare runs and understand performance deltas more quickly

Supporting multiple benchmark families

The copilot was centered on key benchmark flows already present in the platform, such as:

- DPDK Crypto / CryptoPerf-style workloads
- testpmd
- L3fwd / related forwarding workloads

Because **Radheshyam** already understood how these benchmarks were automated, parsed, and compared, he could shape the AI assistance around real operator tasks rather than abstract chatbot use cases.

g) Evaluation, release quality, and production readiness (Radheshyam)

Adding CI evaluation gates for AI quality

A major production contribution from **Radheshyam** was adding measurable quality controls to the AI system.

He introduced golden-set style CI evaluation gates around metrics such as:

- **faithfulness / groundedness**
- **context precision**
- **context recall**
- **citation coverage**
- **tool success rate**
- **tool error rate**
- **p95 latency**

This helped ensure that the copilot did not regress silently as prompts, retrieval behavior, or tooling changed.

Blocking regressions before release

By putting these evaluation gates into CI/CD, **Radheshyam** helped move the project from experimentation toward **disciplined LLMOps / AI platform engineering**.

This was one of the clearest signs that the system was being treated as a production-style service rather than a prototype.

h) Production deployment and operational safeguards (Radheshyam)

Deploying the copilot like an enterprise backend service

To make the system operationally viable, **Radheshyam** helped shape the platform for production-style deployment using:

- **FastAPI**
- **Kubernetes**
- **Helm**
- **HPA**
- **Jenkins**
- **Postgres**
- **Vector DB**
- **S3 / MinIO**

Operational safeguards

The platform included safeguards such as:

- tracing across retrieval and tool calls
- retries for transient failures
- timeouts for dependent services
- circuit breaker style dependency protections
- canary / rollback deployment thinking

These controls were important because the system depended on multiple moving pieces and needed to behave reliably under real usage conditions.

i) Leadership, mentoring, and evolution mindset (Radheshyam)

Technical leadership across both the deterministic and AI layers

One of the strongest aspects of **Radheshyam's** contribution was that he could bridge:

- the original automation platform
- the benchmark domain
- the AI copilot architecture
- the production deployment model

This gave him the ability to make strong cross-cutting decisions such as:

- what should become a shared platform capability

- what should remain rule-driven or tool-driven
- how to keep AI grounded and safe
- how to connect evaluation, observability, and deployment discipline to the AI system

Scaling knowledge for the team

Because **Radheshyam** had already authored AMD-centric documentation and helped onboard teammates into the DPDK domain, he was well-positioned to evolve that knowledge into an AI-assisted platform that could scale expertise beyond a small group of specialists.

3. Impact of Radheshyam's Contribution

Grounded AI assistance for benchmark tuning and regression questions

As a result of **Radheshyam's** work, the platform evolved from being only an automation framework into a more intelligent operator-assistance system.

Engineers could now ask tuning and regression-related questions and receive:

- grounded answers
- contextual evidence
- citations from runs, logs, and guides
- workload-aware responses across AMD server generations

This improved trust and reduced the need to manually stitch together fragmented information.

Faster benchmark analysis and reduced reliance on tribal knowledge

Because the copilot could connect benchmark artifacts, docs, and historical runs in one flow, the team no longer had to depend as heavily on a small set of experienced engineers for interpretation.

This made benchmark reasoning:

- faster
 - more repeatable
 - easier to scale across the team
-

Safer operational workflows through deterministic tool boundaries

By exposing run lookup, log fetching, comparison, and command construction through MCP tools, **Radheshyam** ensured that AI assistance did not weaken operational safety.

The result was a much safer model where:

- commands stayed deterministic
 - tool outputs were auditable
 - execution paths were more consistent
 - the system remained enterprise-friendly
-

Higher release confidence through measurable AI quality gates

The addition of CI evaluation gates around groundedness, citation coverage, tool success, retrieval quality, and latency significantly improved confidence in platform changes.

This helped the team catch issues before release and made the AI layer much more governable.

Stronger platform maturity and production readiness

Because of **Radheshyam's** architecture and engineering decisions, the BenchOps Copilot behaved more like a real internal AI platform capability than a simple RAG demo.

It had:

- structured storage and artifact lineage
- workflow orchestration
- deterministic tool boundaries
- observability
- deployment safeguards
- evaluation gates
- production-style backend behavior

Overall transformation driven by Radheshyam

Overall, **Radheshyam's contribution transformed the benchmark ecosystem in two major stages:**

1. from a fragmented manual process to a structured AMD-centric benchmark automation framework
2. from a deterministic benchmark framework to a grounded, safe, production-style AI BenchOps Copilot

This made the platform significantly more valuable for benchmark planning, tuning guidance, regression analysis, and operational scale.

4. Lessons Learned – Reflections from Radheshyam

AI becomes useful faster when built on top of a strong deterministic platform

One of the biggest lessons for **Radheshyam** was that enterprise AI works best when the underlying operational system is already mature.

Because the team had already built:

- automation
- parsers
- templates
- reporting
- comparison flows
- platform-specific documentation

the AI layer had real structure to build on.

This reduced ambiguity and made the AI system more useful and trustworthy.

Metadata quality is critical for benchmark-domain retrieval

Radheshyam learned that in a domain like performance engineering, semantic search alone is not enough.

Meaningful answers depend on whether the system retrieves the right context for:

- workload
- AMD generation
- source type
- benchmark phase
- run lineage

This reinforced the importance of careful metadata extraction and retrieval filtering.

Safe AI systems need architectural controls, not just prompts

Another major lesson for **Radheshyam** was that AI safety in backend systems cannot depend only on prompt instructions.

It needs hard boundaries such as:

- deterministic tools
- schema validation
- audit logging
- evaluation gates
- approval gates for risky actions
- clear distinction between truth source and generated synthesis

This project strengthened **Radheshyam's** thinking about secure-by-design AI systems.

Evaluation must reflect operational quality, not just answer quality

The project also taught **Radheshyam** that AI quality should be measured using metrics that matter operationally, such as:

- groundedness
- citation coverage
- context precision/recall
- tool success rate
- latency
- release regression detection

This was a key step in moving from experimentation to production-grade AI engineering.

Platform value comes from connecting the pieces well

Finally, **Radheshyam** learned that the real strength of an enterprise AI platform is not any single component by itself.

The value comes from combining:

- deterministic benchmark automation
- retrieval
- tool boundaries
- workflow orchestration
- evaluation

- observability
- deployment discipline

into one coherent system.

5. Skills Demonstrated (for mapping)

Core skills (Experienced level)

- **AI platform architecture** – Defined how RAG, orchestration, tools, storage, evaluation, and deployment fit together into an enterprise benchmark copilot.
- **Backend and system design** – Designed a production-style multi-component backend combining FastAPI, Postgres, vector retrieval, artifact storage, and deterministic tools.
- **Technical judgment** – Made strong decisions about what should remain deterministic versus AI-assisted.
- **Production engineering mindset** – Added observability, evaluation gates, deployment safeguards, and safer workflow controls.

Common skills (Foundation / Strong level)

- **Leadership and ownership** – Took ownership of the platform's evolution from automation to AI assistance.
- **Cross-functional collaboration** – Worked across benchmark engineering, documentation, tooling, reporting, backend architecture, and productionization.
- **Knowledge structuring** – Converted fragmented operational knowledge into structured, retrievable, and reusable platform intelligence.
- **Continuous learning and scale thinking** – Built on deep benchmark domain learning and then evolved the system into a reusable AI platform capability.

Specialist skills

- **RAG / Retrieval Engineering** – LlamaIndex ingestion, normalization, chunking, metadata extraction, embeddings, vector indexing, citation-based retrieval.
 - **LLMOps / Workflow Engineering** – LangChain + LangGraph for intent routing, hybrid retrieval, tool invocation, verification, and response shaping.
 - **MCP / Deterministic Tooling** – Safe exposure of run query, log fetch, run diff, and command builder functions.
 - **Performance engineering domain knowledge** – Applied real benchmark workflow understanding for DPDK Crypto, testpmd, and forwarding benchmarks.
 - **Cloud-native deployment** – Kubernetes, Helm, HPA, rollback thinking, tracing, retries, and dependency protections.
 - **Evaluation engineering** – Faithfulness, context precision/recall, citation coverage, tool reliability, and latency gating in CI/CD.
-